

BibleApp API Contract

v1 draft — Auth, Account, Reading Progress, Highlights & Notes (everything the “user accounts and sync” roadmap item needs)

This is the authoritative source for every request/response shape the :server module will implement in Phases 2–4 of the roadmap. It's a living document — update it before changing a route, not after. The canonical copy is `docs/api_contract.md`; this PDF is a snapshot of it for easy reading/printing.

Conventions

Apply to every endpoint in this document unless stated otherwise.

- **Base path:** `/api/v1`
- **Auth:** every endpoint requires `Authorization: Bearer <access token>` unless marked *Auth: none*. Resource endpoints are implicitly scoped to the authenticated user — nobody can read, edit, or delete another user's data, and the user is never a request field (it comes from the token).
- **IDs:** server-generated UUID strings.
- **Timestamps:** ISO-8601 strings, UTC (e.g. 2026-09-11T14:32:00Z) — never epoch millis.
- **Errors:** every non-2xx response uses the shared `ErrorResponse` shape (below).

Design decisions

Written down once so they don't get re-litigated per-endpoint later.

Auth & sessions

- 1 **Two tokens, not one.** A short-lived access JWT (minutes, validated by the existing `plugins/Security.kt` scaffold) plus a longer-lived opaque refresh token, stored hashed server-side per session/device. A single long-lived JWT can't be revoked before it expires; this can.
- 2 **Refresh token rotation.** Every call to `/auth/refresh` returns a new refresh token and invalidates the one just used. Reuse of an already-used refresh token signals theft, not normal client behavior.
- 3 **Tokens travel in the JSON body, not cookies.** This is a native Android client, not a browser — there's no cookie jar to lean on. The client stores both tokens in encrypted local storage (Phase 6).
- 4 **Login failures are deliberately generic** (`INVALID_CREDENTIALS`, whether the email doesn't exist or the password is wrong) — standard defense against account enumeration.
- 5 **Account deletion requires re-entering the password**, not just a valid access token — a destructive, irreversible action deserves a stronger check than “whoever holds the bearer token.”
- 6 **Account deletion cascades** to every highlight, note, reading-progress record, and refresh token owned by that user. Not optional polish: Google Play's User Data policy requires an in-app path to delete an account and its data for any app that lets users create one.

Reading progress

- 7 **Modeled as a singleton per user** (GET/PUT only — no id, no list). There's exactly one “where I left off,” not a collection of them.
- 8 **The client sends its own readAt timestamp with every update; the server only applies it if readAt is at least as new as the stored updatedAt.** Otherwise a device that was offline for a while and syncs last could overwrite a newer position from another device — a visible regression this one field prevents. A stale write isn't an error; the server just returns its current value unchanged.

Notes & Highlights

- 9 **A highlight or note covers an explicit list of verses** (verses: List<VerseLocation>), not a range — matching the real UX of selecting some verses (contiguous or not) and saving them as one library card.
- 10 **Verses may span multiple chapters or books.** Nothing in the contract restricts a highlight/note to one chapter.
- 11 **versionId lives on the highlight/note itself, not per verse** — mixing versions within one highlight isn't meaningful, since versification differs between translations.
- 12 **Display order is a client concern.** Sort client-side by (bookId, chapter, verse) if the library needs Bible order.
- 13 **Deletions are soft, not hard** (deletedAt: String?). A plain GET returns only active items; updatedSince also returns tombstones, so a syncing device can tell “deleted” apart from “not sent yet.”
- 14 **Highlights are recolor-only after creation; Notes allow editing both text and the verse list.** Delete-and-recreate covers everything else for a highlight.
- 15 **No pagination yet**, but GET .../?bookId=&chapter= now matches if *any* entry in verses is in that book/chapter.
- 16 **Server does not validate verse locations against real per-version verse counts** — the client already has this data (ChapterStructure) and is trusted.
- 17 **verses must be non-empty**, capped at a sane maximum (e.g. 500) as a payload guard.

General

- 18 **No conflict resolution beyond last-write-wins** (reading progress's readAt check is the one deliberate exception). Fine for one person syncing across their own devices.

Shared types

```
@Serializable
data class VerseLocation(
    val bookId: Int,    // 1..66, standard canon numbering
    val chapter: Int,
    val verse: Int
)

@Serializable
data class ErrorResponse(
    val code: String,
    val message: String,
    val fieldErrors: Map<String, String>? = null
)
```

Error codes used in this document

Code	Status	Meaning
VALIDATION_ERROR	400	Malformed body — bad email format, password too short, empty/oversized verses, bookId outside 1..66, etc.
INVALID_CREDENTIALS	401	Login or account-deletion password confirmation failed — same response whether the email doesn't exist or the password is wrong.
UNAUTHORIZED	401	Missing, invalid, or expired access/refresh token.
EMAIL_ALREADY_REGISTERED	409	Registration attempted with an email already in use.
NOT_FOUND	404	No resource with that id — also returned when the id belongs to another user, so ownership is never leaked by a 403.

Auth

```
@Serializable
data class RegisterRequest(val email: String, val password: String)

@Serializable
data class LoginRequest(val email: String, val password: String)

@Serializable
data class RefreshRequest(val refreshToken: String)

@Serializable
data class LogoutRequest(val refreshToken: String)

@Serializable
data class AuthResponse(
    val userId: String,
```

```
    val accessToken: String,  
    val accessTokenExpiresInSeconds: Long,  
    val refreshToken: String  
)
```

POST /api/v1/auth/register

Auth: none

Request: RegisterRequest

Success: 201 Created → AuthResponse

Errors: 400 VALIDATION_ERROR, 409 EMAIL_ALREADY_REGISTERED

POST /api/v1/auth/login

Auth: none

Request: LoginRequest

Success: 200 OK → AuthResponse

Errors: 400 VALIDATION_ERROR, 401 INVALID_CREDENTIALS

POST /api/v1/auth/refresh

Auth: none — the refresh token itself is the credential

Request: RefreshRequest

Success: 200 OK → AuthResponse (new access token and a newly-rotated refresh token)

Errors: 401 UNAUTHORIZED (invalid, expired, or already-rotated refresh token)

POST /api/v1/auth/logout

Auth: required

Request: LogoutRequest

Success: 204 No Content — revokes only that session; other devices stay logged in

Errors: 401 UNAUTHORIZED

Account

```
@Serializable
data class UserResponse(val id: String, val email: String, val createdAt: String)
```

```
@Serializable
data class DeleteAccountRequest(val password: String)
```

GET /api/v1/users/me

Auth: required

Success: 200 OK → UserResponse

Errors: 401 UNAUTHORIZED

DELETE /api/v1/users/me

Auth: required

Request: DeleteAccountRequest

Success: 204 No Content — deletes the account and cascades to all owned highlights, notes, reading progress, and refresh tokens

Errors: 401 UNAUTHORIZED (bad/missing token), 401 INVALID_CREDENTIALS (wrong password confirmation)

Reading Progress

```
@Serializable
data class ReadingProgressResponse(
    val versionId: String,
    val bookId: Int,
    val chapter: Int,
    val verse: Int,
    val updatedAt: String
)
```

```
@Serializable
data class UpdateReadingProgressRequest(
    val versionId: String,
    val bookId: Int,
    val chapter: Int,
    val verse: Int,
    val readAt: String // client's local timestamp for this read event
)
```

GET /api/v1/reading-progress

Auth: required

Success: 200 OK → ReadingProgressResponse

Errors: 401 UNAUTHORIZED, 404 NOT_FOUND (nothing synced yet)

PUT /api/v1/reading-progress

Auth: required

Request: UpdateReadingProgressRequest

Success: 200 OK → ReadingProgressResponse — the new position, or the still-current one if readAt was stale

Errors: 400 VALIDATION_ERROR, 401 UNAUTHORIZED

No DELETE: resetting progress is just another PUT; there's no meaningful "no position" state once one exists.

Highlights

A set of verses (in one version) + a color. The verse list is immutable after creation; only the color can be changed.

```
@Serializable
data class CreateHighlightRequest(
    val versionId: String,
    val verses: List<VerseLocation>,    // non-empty
    val color: Int                      // ARGB, matches UserVerse.highlightColor today
)

@Serializable
data class UpdateHighlightRequest(val color: Int)    // the only editable field

@Serializable
data class HighlightResponse(
    val id: String,
    val versionId: String,
    val verses: List<VerseLocation>,
    val color: Int,
    val createdAt: String,
    val updatedAt: String,
    val deletedAt: String? = null
)

@Serializable
data class HighlightListResponse(val highlights: List<HighlightResponse>)
```

GET /api/v1/highlights

Auth: required

Query: bookId (optional), chapter (optional, requires bookId), updatedSince (optional ISO-8601)

Success: 200 OK → HighlightListResponse (active only, unless updatedSince is set)

Errors: 401 UNAUTHORIZED

POST /api/v1/highlights

Auth: required

Request: CreateHighlightRequest

Success: 201 Created → HighlightResponse

Errors: 400 VALIDATION_ERROR, 401 UNAUTHORIZED

PATCH /api/v1/highlights/{id}

Auth: required

Request: UpdateHighlightRequest

Success: 200 OK → HighlightResponse

Errors: 400 VALIDATION_ERROR, 401 UNAUTHORIZED, 404 NOT_FOUND

DELETE /api/v1/highlights/{id}

Auth: required

Success: 204 No Content (soft delete)

Errors: 401 UNAUTHORIZED, 404 NOT_FOUND

Notes

A set of verses (in one version) + free text. Unlike highlights, both the text and the verse list can be edited in place.

```
@Serializable
data class CreateNoteRequest(
    val versionId: String,
    val verses: List<VerseLocation>,    // non-empty
    val text: String
)

@Serializable
data class UpdateNoteRequest(
    val verses: List<VerseLocation>,    // stays within the same versionId;
    val text: String                    // to change version, delete and recreate
)

@Serializable
data class NoteResponse(
    val id: String,
    val versionId: String,
    val verses: List<VerseLocation>,
    val text: String,
    val createdAt: String,
    val updatedAt: String,
    val deletedAt: String? = null
)

@Serializable
data class NoteListResponse(val notes: List<NoteResponse>)
```

GET /api/v1/notes

Auth: required

Query: bookId (optional), chapter (optional, requires bookId), updatedSince (optional ISO-8601)

Success: 200 OK → NoteListResponse (active only, unless updatedSince is set)

Errors: 401 UNAUTHORIZED

GET /api/v1/notes/{id}

Auth: required

Success: 200 OK → NoteResponse

Errors: 401 UNAUTHORIZED, 404 NOT_FOUND

No equivalent single-highlight GET: highlights are always acted on from the already-loaded reading-view list; notes may be opened in a dedicated editor navigated to independently.

POST /api/v1/notes

Auth: required

Request: CreateNoteRequest

Success: 201 Created → NoteResponse

Errors: 400 VALIDATION_ERROR, 401 UNAUTHORIZED

PUT /api/v1/notes/{id}

Auth: required

Request: UpdateNoteRequest

Success: 200 OK → NoteResponse

Errors: 400 VALIDATION_ERROR, 401 UNAUTHORIZED, 404 NOT_FOUND

DELETE /api/v1/notes/{id}

Auth: required

Success: 204 No Content (soft delete)

Errors: 401 UNAUTHORIZED, 404 NOT_FOUND

Deliberately deferred (not in v1)

- **Password reset / forgot-password flow** — needs an email-sending provider, not set up yet. Until then, a lost password means a lost account.
- **Email verification on register** — same email-infrastructure dependency.
- **Change password / change email** — straightforward later additions; no screen or need for them yet.
- **Multi-session management** (list/revoke other devices) — logout only ends the current device's session.
- **Rate limiting on auth endpoints** — a Phase 5 hardening concern; doesn't change any shape here, just needs enforcing before public launch.

Open items carried over from Notes & Highlights

- Whether highlight color needs validation against a fixed palette vs. accepting any ARGB int.
- Whether verse locations should eventually be validated server-side against real per-version verse counts — needs the server to hold each version's ChapterStructure.
- Whether GET /highlights/{id} is needed once/if a highlight detail view is added.
- How verses is persisted server-side (a jsonb column vs. a child table) is a Phase 3 storage decision — it doesn't change this contract either way.

Source of truth: docs/api_contract.md in the repository. This PDF is generated from it for easy reading — if the two ever disagree, the markdown file wins.